



Least Authority
PRIVACY MATTERS

solana-ed25519

Security Audit Report

Anza Technology

Final Audit Report: 23 June 2026

Table of Contents

[Overview](#)

[Background](#)

[Project Dates](#)

[Review Team](#)

[Coverage](#)

[Target Code and Revision](#)

[Supporting Documentation](#)

[Areas of Concern](#)

[Findings](#)

[General Comments](#)

[System Design](#)

[Code Quality](#)

[Documentation and Code Comments](#)

[Scope](#)

[Specific Issues & Suggestions](#)

[Issue A: Secret Key and Scalar Material Is Copyable, Debug-Printable, and Incompletely Zeroized](#)

[Issue B: Feature-Gated Builds Fail or Mask Broken Public Features](#)

[Issue C: Short-Weierstrass Conversions Can Violate Point and Encoding Invariants](#)

[Issue D: Triple-Base API Has Under-Enforced Contracts](#)

[Issue E: Key Parsing and Verification-Key Validation Are Inconsistent](#)

[Suggestions](#)

[Suggestion 1: Improve Documentation](#)

[Suggestion 2: Improve Test Coverage](#)

[Suggestion 3: Improve Code Quality](#)

[About Least Authority](#)

[Our Methodology](#)

Overview

Background

Anza Technology has requested that Least Authority perform a security audit of solana-ed25519.

Project Dates

- **May 25, 2026 - June 12, 2026:** Initial Code Review (*Completed*)
- **June 16, 2026:** Delivery of Initial Audit Report (*Completed*)
- **June 23, 2026:** Verification Review
- **June 23, 2026:** Delivery of Final Audit Report

Review Team

- Poulami Das, Security / Cryptography Researcher and Engineer
- Mirco Richter, Cryptography Researcher and Engineer
- Burak Atasoy, Project Manager
- Jessy Bissal, Technical Editor

Coverage

Target Code and Revision

For this audit, we performed research, investigation, and review of solana-ed25519 followed by issue reporting, along with mitigation and remediation instructions as outlined in this report.

The following code repository was considered in-scope for the review:

- solana-ed25519:
<https://github.com/anza-xyz/cryptography/tree/master/curve25519/solana-ed25519>
 - Review of conformance with the ed25519 standard and ZIP-215

Specifically, we examined the following Git revision for our initial review:

- 4413a1284ae1d4c7fe942f9d6bafd19f0c9e8361

For the verification, we examined the following Git revision:

- 11a5cf38fb7e11b3ac8d485f8c7820b3db06592e

For the review, this repository was cloned for use during the audit and for reference in this report:

- anza-xyz-cryptography:
<https://github.com/LeastAuthority/anza-xyz-cryptography>

All file references in this document use Unix-style paths relative to the project's root directory.

In addition, any dependency and third-party code, unless specifically mentioned as in scope, were considered out of scope for this review.

Supporting Documentation

The following documentation was available to the review team:

- Website:
<https://www.anza.xyz>

In addition, this audit report references the following documents:

- M. ElSheikh, İ. K. Paksoy, M. Cenk, and M. A. Hasan, "Accelerating EdDSA Signature Verification with Faster Scalar Size Halving." *TCHES* 2025, [EPC+25]
- RFC 8032 – Edwards-Curve Digital Signature Algorithm (EdDSA):
<https://www.rfc-editor.org/info/rfc8032>
- RFC 7748 – Elliptic Curves for Security (Curve25519, X25519, scalar clamping):
<https://www.rfc-editor.org/info/rfc7748>
- RFC 9380 – Hashing to Elliptic Curves (map_to_curve and Elligator):
<https://www.rfc-editor.org/info/rfc9380>
- RFC 9496 – The ristretto255 and decaf448 Groups:
<https://www.rfc-editor.org/rfc/rfc9496.html>
- RFC 5958 – Asymmetric Key Packages (PKCS#8 v1/v2 key encoding):
<https://www.rfc-editor.org/info/rfc5958>
- RFC 8410 – Algorithm Identifiers for Ed25519/X25519 (PKIX):
<https://www.rfc-editor.org/info/rfc8410>
- ZIP-215 – Explicitly Defining and Modifying Ed25519 Validation Rules:
<https://zips.z.cash/zip-0215>
- curve25519-dalek – dalek-cryptography:
<https://github.com/dalek-cryptography/curve25519-dalek>
- Ed25519-zebra – Zcash Foundation:
<https://github.com/ZcashFoundation/ed25519-zebra>

Areas of Concern

Our investigation focused on the following areas:

- Correctness of the implementation;
- Vulnerabilities within each component and whether the interaction between the components is secure;
- Whether requests are passed correctly to the network core;
- Key management, including secure private key storage and management of encryption and signing keys;
- Denial of Service (DoS) and other security exploits that would impact the intended use or disrupt the execution;
- Protection against malicious attacks and other ways to exploit;
- Inappropriate permissions and excess authority;
- Data privacy, data leaking, and information integrity; and
- Anything else as identified during the initial analysis phase.

Findings

General Comments

The Anza cryptography repository contains Rust cryptographic libraries and syscall-facing primitives for the Solana ecosystem, including Curve25519/Ed25519, CUDA acceleration, BN254, BLS12-381, and experimental proof-of-knowledge components. Our audit was limited to the `solana-ed25519` crate, a pure-Rust Curve25519/Ristretto and Ed25519 implementation forked from `curve25519-dalek` and `ed25519-zebra`, extended with ZIP-215-compatible Ed25519 verification and HEEA-accelerated scalar decomposition. In the broader Solana context, this crate provides Ed25519 signing, verification, scalar, and curve operations intended to support Solana cryptographic infrastructure and high-throughput signature-verification use cases.

We compared the `curve25519/solana-ed25519` implementation against ZIP-215, upstream `curve25519-dalek`, upstream `ed25519-zebra`, local and remote README baselines, and the HEEA algorithm from the [\[EPC+25\]](#) paper. We traced Ed25519 verification from challenge computation through scalar parsing, HEEA decomposition, transformed verification, cofactor clearing, serial and AVX2 triple-base MSM execution, and batch-verification randomness. We also reviewed key parsing, import, export, signing, scalar and field arithmetic, Ristretto/Lizard, Montgomery, short-Weierstrass conversion code, feature gating, `serde` behavior, and zeroization as well as secret-exposure surfaces.

We analyzed the code for reachability, attacker control, realistic exploitability, and concrete security impact, with a particular focus on signature unforgeability, reference compatibility, and existing invariants. This included reviewing HEEA/I256 wrapping behavior, `legacy_compatibility::Scalar::from_bits`, debug-vs.-release assertions, non-canonical point encodings, SPKI algorithm handling, `verify_dalek` semantics, `ff/serde` feature behavior, lookup-table/NAF drift, Pippenger/Straus layout performance, `Scalar52` exposure, and short-Weierstrass identity and affine edge cases. In particular, we confirmed that standard verification always provides HEEA with a reduced challenge scalar and did not identify any crafted-input path that would break verifier soundness through the I256 wrapping logic.

We examined feature-gated build behavior and found that documented `serde` and PKCS#8 configurations can fail for downstream users while local CI masks the breakage ([Issue B](#)). We then reviewed key-import and verification-key handling and found that malformed or mislabeled key material can pass through inconsistent parsing, validation, and error-reporting paths ([Issue E](#)). We analyzed short-Weierstrass conversion helpers and found that exceptional or invalid affine inputs can violate point-validity and encoding round-trip assumptions ([Issue C](#)). We further evaluated triple-base verification support and found under-enforced scalar-size contracts ([Issue D](#)). We also assessed secret-material lifecycle handling and found that signing keys and scalar values are copyable, accidentally exposable through generic byte and debug interfaces, and incompletely zeroized across signing, serialization, PKCS#8 export, reduction, inversion, and backend scratch paths ([Issue A](#)).

System Design

Our team reviewed `curve25519/solana-ed25519` and found that it has been well-designed, with security at the forefront of its architecture. This is evident in its adherence to Rust best practices and its foundation on the well-audited `curve25519-dalek` cryptographic library.

Dependencies

Running `cargo audit` revealed no vulnerabilities in the in-scope components. Our team therefore did not identify any dependency-related security issues.

Code Quality

We performed a manual review of the repositories in scope and found the code to be well-organized and aligned with development best practices. However, we identified several areas where code quality can be improved ([Suggestion 3](#)).

Tests

The in-scope repository includes generally strong test coverage. However, we recommend that new functionality be tested more thoroughly ([Suggestion 2](#)).

Documentation and Code Comments

The project documentation provided was generally sufficient. However, a substantial number of docstrings, headers, and READMEs describe behavior that is not implemented in the code ([Suggestion 1](#)). In addition, inline comments are sufficient and accurate for the core cryptographic math, except in a few areas where comments are missing at points where the local code diverges from upstream forked code or the implemented papers.

Scope

The scope of this review was sufficient and included all security-critical components.

Specific Issues & Suggestions

We list the issues and suggestions identified during the review in descending order of severity. In most cases, remediation of an issue is preferable, but mitigation is suggested as another option for cases where a trade-off could be required.

ISSUE / SUGGESTION	SEVERITY	STATUS
Issue A: Secret Key and Scalar Material Is Copyable, Debug-Printable, and Incompletely Zeroized	Medium	Partially Resolved
Issue B: Feature-Gated Builds Fail or Mask Broken Public Features	Medium	Resolved
Issue C: Short-Weierstrass Conversions Can Violate Point and Encoding Invariants	Medium	Resolved
Issue D: Triple-Base API Has Under-Enforced Contracts	Medium	Resolved
Issue E: Key Parsing and Verification-Key Validation Are Inconsistent	Low	Resolved
Suggestion 1: Improve Documentation	Informational	Partially Resolved
Suggestion 2: Improve Test Coverage	Informational	Resolved
Suggestion 3: Improve Code Quality	Informational	Resolved

Issue A: Secret Key and Scalar Material Is Copyable, Debug-Printable, and Incompletely Zeroized

Location

[curve25519/solana-ed25519/src/ed_sigs/signing_key.rs#L62](#)

[curve25519/solana-ed25519/src/ed_sigs/signing_key.rs#L105](#)

[curve25519/solana-ed25519/src/ed_sigs/signing_key.rs#L266](#)

[curve25519/solana-ed25519/src/ed_sigs/signing_key.rs#L348](#)

[curve25519/solana-ed25519/src/scalar.rs#L162](#)

[curve25519/solana-ed25519/src/scalar.rs#L253](#)

[curve25519/solana-ed25519/src/scalar.rs#L859](#)

[curve25519/solana-ed25519/src/field.rs#L184](#)

[curve25519/solana-ed25519/src/backend/serial/u64/scalar.rs#L30](#)

Synopsis

The crate exposes Ed25519 signing-key and scalar secrets through ordinary Rust convenience surfaces while also leaving the zeroize feature incomplete. SigningKey and Scalar are Copy, so secret values are duplicated implicitly; Scalar and Scalar52 print raw values through Debug; SigningKey implements AsRef<[u8]> over the raw seed; serde and PKCS#8 paths copy seed bytes through ordinary buffers; and several signing, reduction, inversion, and backend scratch buffers are dropped without being wiped. The result is not a cryptographic forgery by itself, but a secret-lifecycle failure that can disclose seeds, expanded signing scalars, nonces, or scalar-derived intermediates to logs, serialized byte streams, heap reuse, stack residue, or memory forensics.

Preconditions

The vulnerable paths are relevant when applications use solana-Ed25519 with private signing keys, secret scalars, or secret-derived points and either rely on the zeroize feature for memory hygiene or pass these values through generic byte, serde, logging, Debug, or export infrastructure. Some manifestations require optional features such as zeroize, serde, pkcs8, alloc, rand_core, or an AVX2 runtime backend. Exploitation generally requires the attacker to obtain logs, telemetry, serialized output, process memory, crash dumps, or memory reused after secret-bearing operations.

Likelihood

Low.

The disclosure surfaces are public, idiomatic, and easy to reach accidentally, especially AsRef<[u8]>, Debug, serde, Copy, and standard scope exit. Successful exploitation still depends on access to logs, exported bytes, memory snapshots, or a process context that can observe stale memory.

Impact

High.

Disclosure of an Ed25519 seed or expanded signing scalar compromises the signing key. Disclosure of a per-message nonce scalar can also reveal the signing scalar. Even partial scalar-derived residues can materially weaken the expected protection provided by memory hygiene controls.

Severity
Medium.

Technical Details

SigningKey stores the raw seed, expanded scalar, signing prefix, and verification key, but derives Copy and only zeroizes the particular instance passed to zeroize. Its AsRef<[u8]> implementation returns the raw seed, allowing generic byte consumers such as encoders, logging adapters, file or network writers, and serde helpers to process secret material without an explicit secret-bearing API. The serde helper converts a raw [u8; 32] into a SigningKey and converts a SigningKey back into raw seed bytes, while the PKCS#8 encoders build local final_key buffers containing the seed and return without wiping those buffers.

Scalar has the same lifecycle problem: it derives Copy, implements Debug by printing raw bytes, and is used for expanded signing scalars, per-signature nonces, random scalars, and blinding-like protocol values. Signing computes the nonce scalar r from the secret signing prefix and message, then combines r with self.s, but the nonce scalar and local SHA-512 reduction buffer are not explicitly zeroized.

Remediation

We recommend the following steps to address this issue:

- Removing Copy from SigningKey, Scalar, and other secret-bearing representations, and adding ZeroizeOnDrop or equivalent Drop implementations for secret types.
- Redacting Debug for Scalar, removing or renaming SigningKey::AsRef<[u8]> and ambiguous raw-seed accessors, and exposing secret bytes only through intentionally named APIs with a clear cleanup contract.
- Replacing serde raw-byte helpers with a nominal seed type, and wrapping PKCS#8 temporary buffers and SHA-512/reduction buffers in Zeroizing.

Status

The Anza team has fixed several important parts of the issue: SigningKey is no longer Copy, it is zeroized on drop when the zeroize feature is enabled, SigningKey::AsRef<[u8]> was removed, scalar Debug output is redacted, and several signing, PKCS#8, SHA-512, reduction, and inversion temporaries are now wiped. The all-features test suite also passes.

However, the remediation remains incomplete. Scalar and Scalar52 remain Copy, older raw-seed accessors such as SigningKey::to_bytes and SigningKey::as_bytes still exist as aliases, serde still uses a raw [u8; 32] helper rather than a nominal seed type, and some scalar-derived backend scratch buffers remain unwiped.

Verification

Partially Resolved.

Issue B: Feature-Gated Builds Fail or Mask Broken Public Features

Location

[curve25519/solana-ed25519/Cargo.toml#L25-L32](#)

[curve25519/solana-ed25519/Cargo.toml#L60-L83](#)

[curve25519/solana-ed25519/src/ed_sigs/signing_key.rs#L256-L310](#)

[curve25519/solana-ed25519/src/ed_sigs/verification_key.rs#L118-L120](#)

[curve25519/solana-ed25519/src/ed_sigs/verification_key.rs#L261-L285](https://github.com/solana-labs/solana/blob/master/src/ed_sigs/verification_key.rs#L261-L285)

Synopsis

The `serde` and `pkcs8` feature definitions do not enable the dependency features required by the code they expose. Local workspace checks can hide the `serde` breakage through dev-dependency feature unification, while `docs.rs` is configured to build all features and therefore reaches the broken path.

Preconditions

The issue appears when downstream users enable `serde` or `pkcs8` in standard dependency builds, especially without the crate's dev-dependency graph or without a broader feature such as `pem` that incidentally activates missing upstream support.

Likelihood

High.

Impact

Low.

Downstream users can be blocked from building documented feature combinations, and maintainers can receive false CI signals that those feature combinations are releasable. The `docs.rs` failure also removes the published API documentation for consumers.

Severity

Medium.

Technical Details

The `serde` dependency is optional, but the feature table does not explicitly enable `serde/derive` even though several types use `#[cfg_attr(feature = "serde", derive(...))]`. The `pkcs8` feature enables `dep:pkcs8`, `ed25519/pkcs8`, and `local_alloc`, but not upstream `pkcs8/alloc`, while the gated code imports allocation-backed PKCS#8 and DER document APIs. This combination causes consumer builds to fail and allows workspace all-target checks to provide false confidence.

Remediation

We recommend making the public feature entries explicit and self-contained, for example by adding `serde/derive` to `serde` and `pkcs8/alloc` to `pkcs8`. This includes adding CI jobs that check each public feature without dev-dependencies and run all-features documentation builds before publication.

Status

The Anza team has resolved the feature-gating failure. The `serde` and `pkcs8` fixes are now merged, and the current feature table renders both public features self-contained: `serde` enables `serde/derive`, and `pkcs8` enables `pkcs8/alloc` together with the required local and `ed25519` PKCS#8 support.

Verification

Resolved.

Issue C: Short-Weierstrass Conversions Can Violate Point and Encoding Invariants

Location

[curve25519/solana-ed25519/src/short_weierstrass.rs#L10-L25](https://github.com/solana-labs/solana/blob/master/src/short_weierstrass.rs#L10-L25)

[curve25519/solana-ed25519/src/short_weierstrass.rs#L33-L89](#)

[curve25519/solana-ed25519/src/short_weierstrass.rs#L91-L113](#)

[curve25519/solana-ed25519/src/short_weierstrass.rs#L141-L153](#)

Synopsis

The short-Weierstrass helper type does not consistently preserve the invariant that converted points and byte encodings are valid and round-trippable. Invalid affine values can be converted into `EdwardsPoint`, the Edwards order-2 point does not round-trip, and the all-zero affine encoding is silently accepted as identity.

Preconditions

Applications must use the short-Weierstrass conversion or byte-parsing helpers with externally supplied, mutated, or exceptional points.

Likelihood

Medium.

Impact

Medium.

Invalid or exceptional short-Weierstrass inputs can produce non-round-tripping encodings or invalid Edwards representations that subsequent code may treat as valid curve points. This can cause incorrect arithmetic, hidden input-normalization bugs, and unsafe assumptions in higher-level protocols.

Severity

Medium.

Technical Details

`SwPoint::to_edwards` rejects only a few exceptional denominator cases before constructing an `EdwardsPoint`. It does not first require `self.is_on_curve()`. `SwPoint::from_edwards` can produce a valid short-Weierstrass point with zero affine `y` from the Edwards order-2 point, but `to_edwards` rejects all such points. `from_affine_le_bytes` also special-cases `[0; 32]`, `[0; 32]` as the identity, even though `to_affine_le_bytes` does not emit that representation.

Remediation

We recommend validating affine points before converting them to Edwards form, removing or documenting the all-zero identity encoding, and adding tests for off-curve affine inputs, the Edwards order-2 point, and identity byte round-trips.

Status

The Anza team has resolved the short-Weierstrass invariant issue. `SwPoint::to_edwards` now rejects affine coordinates that are not on the short-Weierstrass curve before constructing an `EdwardsPoint`, and it rejects the exceptional `y == 0` case where the birational map is undefined. In addition, the Edwards order-2 exceptional case is documented.

The affine byte encoding behavior is consistent: the all-zero pair is explicitly reserved as the identity encoding, `to_affine_le_bytes` emits that representation for identity, and `from_affine_le_bytes` accepts it as identity while rejecting off-curve non-identity inputs.

Verification

Resolved.

Issue D: Triple-Base API Has Under-Enforced Contracts

Location

[curve25519/solana-ed25519/src/edwards.rs#L1085-L1106](#)

[curve25519/solana-ed25519/src/backend.rs#L239-L258](#)

[curve25519/solana-ed25519/src/backend/serial/scalar_mul/vartime_triple_base.rs#L40-L104](#)

[curve25519/solana-ed25519/src/backend/vector/scalar_mul/vartime_triple_base.rs#L48-L117](#)

Synopsis

The public triple-base API accepts full scalars while the implementation assumes 128-bit operands.

Preconditions

The highest-impact path requires callers to pass full-size `Scalar` values to the public triple-base routine.

Likelihood

Medium.

Impact

Medium.

Callers can receive silently incorrect group arithmetic in release builds when full-size scalars reach the public triple-base API.

Severity

Medium.

Technical Details

The triple-base wrapper states that it is optimized for 128-bit `a1` and `a2`, but the backends enforce this only with `debug_assert!`. Release builds can silently compute with truncated high bits.

Remediation

We recommend making the triple-base precondition checked in all builds or keeping the helper `crate-private`.

Status

The Anza team has updated the public triple-base API to accept full-width scalars safely. It checks whether `a1` and `a2` fit in 128 bits. If either does not, it falls back to general scalar multiplication instead of using the optimized 128-bit path. In addition, the optimized backend routine is now `crate-private` and documented as `prechecked`, so the under-enforced public contract is no longer exposed to callers.

Verification

Resolved.

Issue E: Key Parsing and Verification-Key Validation Are Inconsistent

Location

[curve25519/solana-ed25519/src/ed_sigs/signing_key.rs#L275-L295](#)

[curve25519/solana-ed25519/src/ed_sigs/verification_key.rs#L99-L167](#)

[curve25519/solana-ed25519/src/ed_sigs/verification_key.rs#L230-L244](#)

[curve25519/solana-ed25519/src/ed_sigs/verification_key.rs#L277-L285](#)

[curve25519/solana-ed25519/src/ed_sigs/batch.rs#L132-L194](#)

Synopsis

Public key-import paths mix panicking parses, missing SPKI algorithm checks, length-only byte wrappers, and misleading batch-verification errors. These are separate manifestations of the same weak boundary between encoded key bytes and validated Ed25519 key material.

Preconditions

The affected paths are reached when applications parse PKCS#8 or SPKI DER, or queue externally supplied verification-key bytes for batch verification.

Likelihood

Medium.

Impact

Low.

Malformed or mislabeled key material can cause availability-sensitive parsing paths to panic, be accepted under the incorrect algorithm label, or surface misleading verification errors. This weakens key-ingestion defenses and makes failures harder to attribute correctly.

Severity

Low.

Technical Details

`SigningKey::from_pkcs8_der` and the SPKI public-key helpers unwrap fallible operations on caller-controlled data despite returning `Result`. The SPKI path extracts the `BIT STRING` without checking that the algorithm identifier is Ed25519 with absent parameters. `VerificationKeyBytes` is documented as a refinement type, but its constructors accept arbitrary 32-byte strings and defer decompression until later. The batch verifier then reports a malformed queued verification key as `InvalidSignature`, while the single-key path reports the same condition as `MalformedPublicKey`.

Remediation

We recommend replacing unwraps with explicit error mapping, validating SPKI algorithm identifiers and parameters, and clarifying whether `VerificationKeyBytes` is checked or unchecked encoded key material. Batch errors should also be aligned with the single-key path, with malformed DER, wrong-OID, wrong-length, invalid-point, and malformed-batch regression tests.

Status

The Anza team has resolved the inconsistent key parsing and verification-key validation issue. PKCS#8 private-key parsing no longer unwraps fallible operations on caller-controlled data, and malformed private

DER now returns an error instead of panicking. SPKI public-key parsing validates the Ed25519 algorithm identifier and rejects incorrect OIDs, unexpected parameters, malformed key bytes, and undecodable public keys through explicit error paths.

The semantics of `VerificationKeyBytes` were also clarified: it is documented as a length-checked encoded byte container, not a fully validated verification key. Validation occurs when converting to `VerificationKey`. Additionally, batch verification now aligns with the single-key path by returning `MalformedPublicKey` for malformed queued verification keys instead of misreporting them as `InvalidSignature`.

Verification

Resolved.

Suggestions

Suggestion 1: Improve Documentation

Location

[curve25519/solana-ed25519/src/scalar.rs#L532-L561](#)

[curve25519/solana-ed25519/src/traits.rs#L423-L438](#)

[curve25519/solana-ed25519/src/ed_sigs/verification_key.rs#L8-L14](#)

[curve25519/solana-ed25519/src/ed_sigs/verification_key.rs#L41-L97](#)

[curve25519/solana-ed25519/src/ed_sigs/verification_key.rs#L400-L430](#)

[curve25519/solana-ed25519/src/backend/serial/u64/field.rs#L58-L73](#)

Synopsis

Although the codebase contains substantial documentation, we identified the following locations where the documentation was incorrect, outdated, or misleading:

- The description of `heea_decompose` in `traits.rs` does not match its underlying implementation.
- The comment on `heea_verify` in `verification_key.rs` is outdated.
- The documentation for `verify_dalek` does not explicitly describe that `verify_dalek_prehashed` rejects all-zero verification-key bytes and a legacy blacklist of R encodings before performing its dalek-style byte comparison. As a result, the method is not strictly byte-exact dalek verification.
- The documentation for `heea_decompose` does not state that it is a variable-time algorithm, which may lead to API misuse by callers expecting a constant-time algorithm.
- The non-reducing `FieldElement51` operators (`AddAssign` and `Add`) in the serial backend do not include documentation comments about the limb-bound discipline they rely on: limbs may grow only to $< 2^{54}$ between reductions (approximately three reduced summands before multiplication, squaring, or reduction). The ceiling is stated on the struct and enforced with `debug_assert!` inside `mul` and `pow2k`, but this producer-side rule is left implicit, unlike the

AVX2 backend (FieldElement2625x4), which documents per-operation precondition and postcondition bounds.

Mitigation

We recommend addressing the documentation issues listed above.

Status

The Anza team has resolved all suggestions above except the last one regarding FieldElement51 operators, which would require an update to the upstream code.

Verification

Partially Resolved.

Suggestion 2: Improve Test Coverage

Location

[curve25519/solana-ed25519/src/backend/vector/scalar_mul/vartime_triple_base.rs#L59](#)

[curve25519/solana-ed25519/src/short_weierstrass.rs](#)

[curve25519/solana-ed25519/src/traits.rs](#)

Synopsis

While overall test coverage is high, the current suite under-exercises several fork-new functions, particularly their negative and error paths and, in some cases, their happy paths. The following functionalities should be covered for both positive and negative inputs:

- `Mul_128_128_256` implements HEEA triple-base multiscalar multiplication. The AVX2/vector implementation has no dedicated tests, yet it is the default verification path on x86-64, Solana's production platform. Only the serial variant is tested. In addition, neither variant is tested with out-of-range scalars ($a1/a2 \geq 2^{128}$), where the function silently truncates high bits in release builds. Tests should assert equivalence to the naive result for both backends and defined behavior (either a correct result or error) for out-of-range inputs.
- `SwPoint::from_affine_le_bytes` is the short-Weierstrass deserialization function and the actual input path for the GPU/CUDA MSM. It is never executed by any test, so neither valid decoding nor rejection of malformed, off-curve, or noncanonical byte inputs is verified. Positive tests should cover well-formed point round-trips. Negative tests should verify that invalid encodings are rejected rather than silently accepted as invalid points.
- `SwPoint::to_affine_le_bytes` is the matching serialization function and is also never executed. It should be tested jointly with `from_affine_le_bytes` for round-trip stability, such that `(to(from(x)) == x)`, which is the property the GPU MSM relies on.
- `SwPoint::identity` is the identity constructor for the new short-Weierstrass type and is never exercised. The identity is a special case that add and `to_edwards` branch on explicitly, so it should be tested to confirm that those special-case paths behave correctly. Positive tests should verify that the identity behaves as the additive neutral element. Negative tests should verify that it does not round-trip incorrectly through `to_edwards`, in connection with the order-2 point gap.

- `Verifier::verify`, the `signature::Verifier` trait implementation on `VerificationKey`, is the ecosystem-facing verification entry point used by generic Ed25519 consumers and is never executed by any test. Positive tests should verify that a valid signature verifies. Negative tests should verify that an invalid or malformed signature is rejected, matching the inherent `verify` method's behavior.
- `Verify_zebra_prehashed` is the core ZIP-215 verification routine. The happy path is covered, but its rejection paths are untested. A noncanonical scalar s , where $s \geq \ell$, and an undecodable R point must both cause rejection. These negative cases are consensus-critical. A regression that stopped rejecting them would accept invalid signatures.

Mitigation

We recommend improving test coverage as described above.

Status

The Anza team has improved test coverage by adding the recommended test cases.

Verification

Resolved.

Suggestion 3: Improve Code Quality

Location

[curve25519/solana-ed25519/src/short_weierstrass.rs#L115-L139](#)

[curve25519/solana-ed25519/src/edwards.rs#L1482-L1486](#)

[curve25519/solana-ed25519/src/edwards.rs#L740-L765](#)

[curve25519/solana-ed25519/src/backend/vector/avx2/edwards.rs#L308-L347](#)

[curve25519/solana-ed25519/src/window.rs#L201-L276](#)

[curve25519/solana-ed25519/src/window.rs#L35-L116](#)

Synopsis

We identified the following areas for code quality improvement:

- `EdwardsPoint::random` fills compressed-point bytes from `R: RngCore` rather than requiring `CryptoRng`, unlike peer random APIs. We recommend tightening the inherent random-point RNG bound to `CryptoRng + RngCore`.
- `SwPoint::add` implements addition as an inherent method, so `p + q` and generic `Add`-bounded code are unsupported for this public point type. We recommend either implementing standard arithmetic traits for `SwPoint` or renaming the inherent addition method to avoid implying `Add` compatibility.
- The serial triple-base path documents width-8 NAF for the basepoint term under precomputed tables but computes width 5, unlike the vector path. Lookup-table construction logic is duplicated across serial and AVX2 code, increasing the chance of future backend divergence. We recommend aligning the serial and vector NAF-width choices, consolidating duplicated lookup-table construction, and adding release-mode and cross-backend regression tests.

Mitigation

We recommend improving the code quality items noted above.

Status

The Anza team has resolved the three code quality improvement points mentioned above by requiring `CryptoRng + RngCore` for `EdwardsPoint::random`, implementing `core::ops::Add` for `SwPoint` to include `+` operator support, and reducing duplicated lookup-table construction logic across serial/vector backends.

Verification

Resolved.

About Least Authority

Founded in 2011 and based in Berlin, Germany, since 2016, Least Authority provides security consulting for privacy-preserving technologies and decentralized systems. We are committed to transparency, openness, and advancing secure digital infrastructure.

We believe that people have a fundamental right to privacy and that secure solutions enable people to use the internet and other connected technologies more freely. Our security consulting services help teams make their solutions more resistant to unauthorized access and unintended system manipulation. We support teams from the design phase through production launch and beyond.

Our team has experience reviewing code for security vulnerabilities and specific attack vectors in multiple languages, including Rust, Go, Python, C, C++, Solidity, Cairo, Circom, Noir, Haskell, OCaml, TypeScript, Swift, ZoKrates, and others. We have reviewed implementations of cryptographic protocols and distributed system architecture in blockchains, payments, smart contracts, zero-knowledge protocols, and consensus protocols. Additionally, we use various tools to scan code and networks and build custom tools as necessary.

For more information about our security consulting, please visit:

<https://leastauthority.com/security-consulting>.

Our Methodology

We use a transparent review process that provides the project team visibility into our findings, assumptions, and remediation considerations throughout the audit. The goals of our security audits are to improve the quality of the systems we review and support sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Code Review

In manually reviewing all of the code, we identify potential issues, including those related to code logic, error handling, cryptographic implementations, and correctness with respect to specifications, standards, and best practices. We also evaluate areas where more defensive programming could reduce the risk of future mistakes and accelerate future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

When appropriate, we also use artificial intelligence (AI) to support the efficiency of our security audits without reducing rigor or reliability. AI-assisted outputs are carefully guided, reviewed, and validated by our team so that conclusions remain accurate and defensible. This supports stronger audit decisions, not only faster review.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and white box penetration testing, supplemented by various tools, including AI, that support the code review process. Discussions with the development team help clarify their vision for the software. Installing and using the relevant software helps us understand user interactions and roles, while also informing our analysis of threat models and attack surfaces. Design documentation, relevant papers, other audit results, similar projects, source code dependencies, and open issue tickets may also be reviewed to inform the analysis. We

identify potential vulnerabilities, and, when appropriate, create issue entries. For each entry, we follow the issue investigation and remediation process described below.

Documenting Results

We follow a conservative and transparent process for analyzing potential security vulnerabilities and supporting successful remediation. Whenever a potential issue is identified, we raise it with the client through a dedicated secure communication channel to discuss its validity and impact, to the extent these can be determined at that stage. This process allows potentially relevant findings to be reviewed with the client early, before they are documented in greater detail in the initial audit report. For each confirmed issue, we assess impact and feasibility and use those ratings to determine the overall severity, following the [OWASP Risk Severity Matrix](#).

Suggested Solutions

We identify immediate and comprehensive mitigations available to live deployments and then suggest remediation requirements for future releases. When code is already deployed in production, we include mitigation strategies in the report to provide interim support until appropriate remediation is available. The mitigation and remediation recommendations should be reviewed by the development team and deployment engineers. After delivering the Initial Audit Report, we remain available to support implementation of the proposed mitigations and remediations before the verification review.

Development teams should address findings in an order and with an urgency that prioritizes the security of their users. Findings should be addressed as soon as possible, but remediation should be carried out pragmatically and should not cause unnecessary disruption to project timelines.

Although issues must be handled on a case-by-case basis, we work toward a resolution timeline that balances the impact on users and the needs of the project team.

Resolutions & Publishing

Once the findings are comprehensively addressed, we complete a verification review to assess whether the issues and suggestions have been sufficiently addressed. When this analysis is complete, we update the report and provide a Final Audit Report that can be published in full. If any critical issues remain, we recommend that the Final Audit Report not be published and that users and other stakeholders be alerted to the impact. We encourage all findings to be addressed and the Final Audit Report to be shared publicly to support transparency and advance security knowledge within the industry.